Maitri — AI-Powered Exercise Rehabilitation Platform

Complete Project Documentation

> **Version:** 1.0.0 | **Date:** May 2026 | **Platform:** Windows 11 / Cross-platform

---

Table of Contents

---

1. Hardware Requirements

Minimum Specifications

Component	Minimum	Recommended
CPU	Intel Core i5 (8th gen) / AMD Ryzen 5 3600	Intel Core i7 (10th gen+) / AMD Ryzen 7 5800X
RAM	8 GB	16 GB
GPU	Integrated / None required	NVIDIA GTX 1060 (for future GPU acceleration)
Storage	5 GB free space	10 GB free space (SSD preferred)
Webcam	720p @ 30 FPS (USB 2.0)	1080p @ 30 FPS (USB 3.0)
Display	1280 × 720	1920 × 1080 or higher
Internet	Required only for Gemini AI rehab recommendations	Stable broadband (10 Mbps+) for low-latency API calls
OS	Windows 10 64-bit	Windows 11 64-bit

Camera Placement Guidelines

Exercise	Camera Position
Squats	~2 m in front at head height; side or 45° angle
Push-ups	Floor level, ~1–1.5 m to the side (side-profile view)
Planks	Floor level, ~1 m to the side (side-profile view)

> **Note:** MediaPipe Pose model runs on CPU. A modern multi-core CPU (4+ cores) is essential for maintaining 30 FPS webcam throughput without dropped frames.

---

2. Software Requirements

Backend (Python)

Package	Version	Purpose
Python	3.11.9	Runtime
mediapipe	0.10.14	Human pose estimation (33 landmarks)
opencv-python	4.13.0.92	Frame capture, decoding, rendering
numpy	2.4.4	Vectorised geometric math
fastapi	0.136.1	REST + WebSocket API server
uvicorn	0.46.0	ASGI server runner
websockets	16.0	WebSocket protocol support
pyttsx3	2.99	Text-to-speech (standalone/desktop mode)
google-generativeai	0.8.6	Gemini AI for rehab recommendations
python-dotenv	1.2.2	Environment variable management
pydantic	2.13.4	Request/response data validation
starlette	1.0.0	ASGI framework (FastAPI dependency)
pywin32 / comtypes	311 / 1.4.16	Windows SAPI5 COM for pyttsx3 TTS thread

Frontend (Node / React)

Package	Version	Purpose
Node.js	18+ LTS	JavaScript runtime
React	19.2.6	UI component framework
TypeScript	~6.0.2	Type-safe JavaScript
Vite	8.0.12	Development server & build tool
Tailwind CSS	4.3.0	Utility-first styling
Recharts	3.8.1	Real-time sparkline analytics charts
Lucide React	1.14.0	Icon set (Activity, Gauge, etc.)
clsx / tailwind-merge	Latest	Conditional class utilities

External APIs & Services

Service	Purpose	Auth
Google Gemini 2.5 Flash	AI-driven physiotherapy rehab recommendations	`GEMINI_API_KEY` in `.env`
Browser Web Speech API	Client-side text-to-speech coaching cues	None (native browser)
MediaDevices (getUserMedia)	Browser webcam access	User permission required

Environment Configuration

Backend (`.env` in project root):

```
GEMINI_API_KEY=your-key-here
```

Frontend (`frontend/.env.local`):

```
VITE_BACKEND_URL=http://localhost:8000
VITE_WS_URL=ws://localhost:8000/ws
```

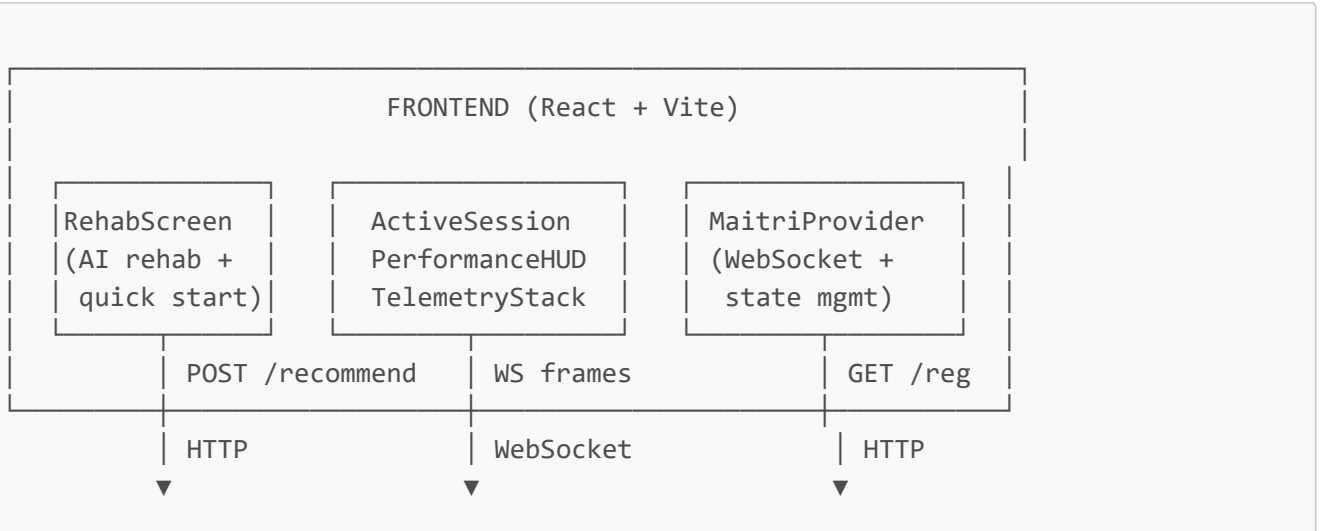
---

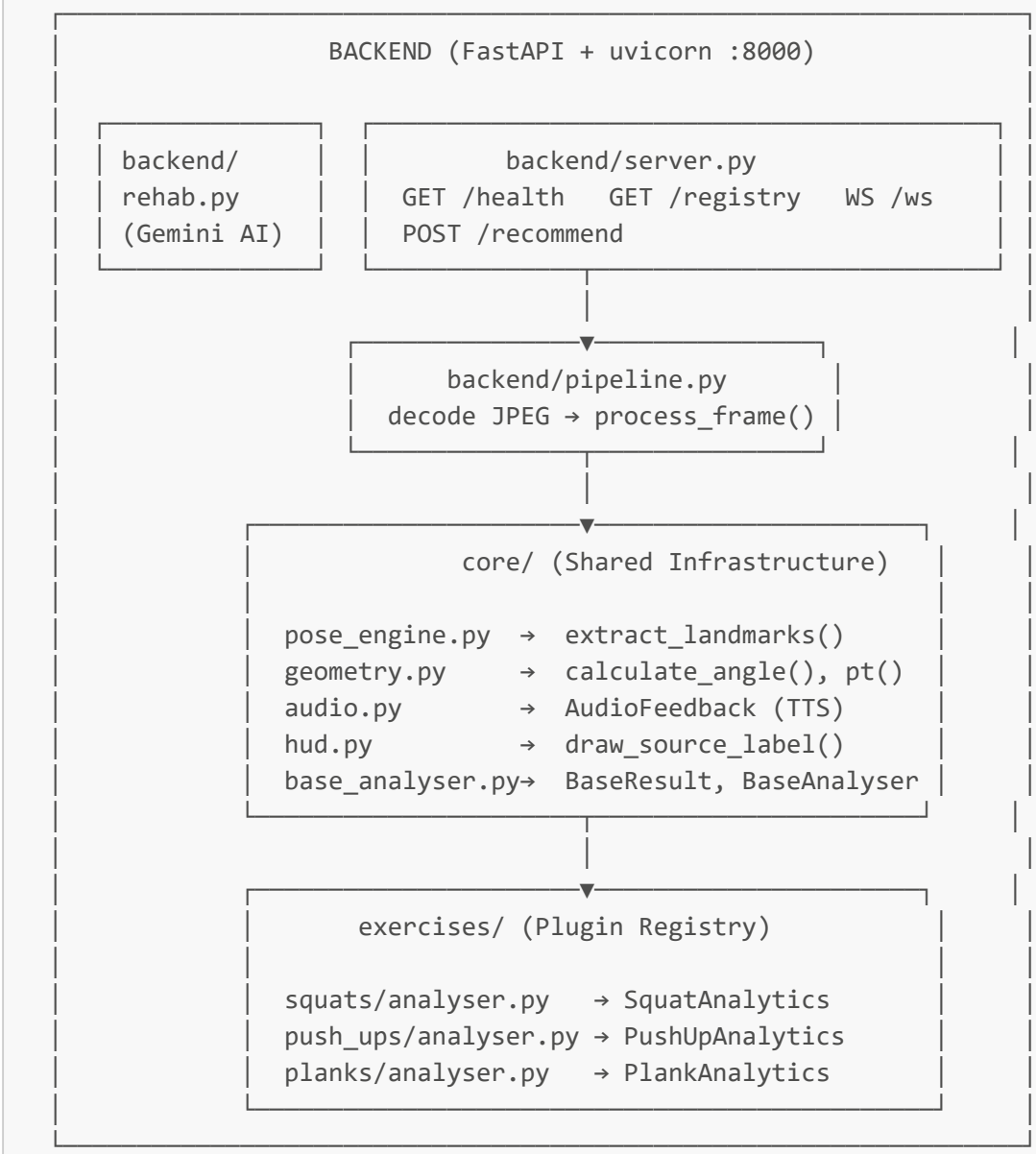
3. System Architecture

High-Level Overview

Maitri is a **client–server** architecture with a complete separation of concerns:

- The **Frontend** (React SPA) handles user interaction, webcam capture, skeleton visualisation, and audio cues via the Web Speech API.
- The **Backend** (FastAPI + uvicorn) handles computer vision, AI pose estimation, exercise analysis logic, and Gemini AI calls.
- Communication is via **HTTP REST** (for registry and rehab recommendations) and **WebSocket** (for real-time frame streaming at ~10 FPS).





Package Layout

```
maitri/
│
├── main.py                    # Standalone desktop entry point (tkinter
launcher)
├── requirements.txt           # Pinned Python dependencies
├── .env                       # GEMINI_API_KEY (not committed)
│
├── core/                      # Shared infrastructure — exercise-agnostic
│   ├── pose_engine.py         # MediaPipe wrapper; Point3D, PoseLandmarks
dataclasses
│   ├── geometry.py            # calculate_angle(), pt(), signed_body_deviation()
│   ├── audio.py               # Non-blocking pyttsx3 TTS (daemon thread)
│   ├── hud.py                 # HUD primitives: draw_source_label(),
resize_to_window()
│   └── base_analyser.py       # BaseResult, BaseAnalyser ABC (plugin contract)
│
```

```
├── exercises/                  # One sub-package per exercise
│   ├── __init__.py             # EXERCISE_REGISTRY — only file to edit for new
exercises
│   ├── squats/
│   │   ├── analyser.py         # SquatResult + SquatAnalytics
│   │   └── hud.py              # draw_squat_hud()
│   ├── push_ups/
│   │   ├── analyser.py         # PushUpResult + PushUpAnalytics
│   │   └── hud.py              # draw_pushup_hud()
│   └── planks/
│       ├── analyser.py         # PlankResult + PlankAnalytics
│       └── hud.py              # draw_plank_hud()
│
├── backend/                    # FastAPI server (client-server mode)
│   ├── server.py               # REST + WebSocket endpoints
│   ├── pipeline.py             # Per-frame processor (JPEG → JSON payload)
│   └── rehab.py                # Gemini AI recommendation engine
│
├── ui/                         # Standalone launcher (testing / desktop mode
only)
│   └── launcher.py             # pick_exercise(), pick_source() tkinter dialogs
│
└── frontend/                   # React + TypeScript SPA
    ├── .env.local               # VITE_BACKEND_URL, VITE_WS_URL
    ├── src/
    │   ├── App.tsx              # Root: routes between RehabScreen ⇔
ActiveSession
    │   ├── context/
    │   │   └── MaitriStreamContext.tsx  # WebSocket manager + global state
    │   ├── components/
    │   │   ├── RehabScreen.tsx    # Landing page: AI rehab plan + quick start
    │   │   ├── ActiveSession.tsx  # Session wrapper: HUD + Telemetry layout
    │   │   ├── PerformanceHUD.tsx # Webcam feed + skeleton canvas overlay
    │   │   └── TelemetryStack.tsx # Rep counter, phase indicator, metric gauges,
chart
    │   └── types/
    │       └── maitri.ts          # TypeScript interfaces: MaitriFrame,
MaitriResult, etc.
    └── package.json
```

Data Structures

Backend — `PoseLandmarks` (Python)

```python
@dataclass
class Point3D:
    x: float   # normalised [0–1], horizontal
    y: float   # normalised [0–1], vertical (0 = top)
    z: float   # relative depth estimate

@dataclass
```

```
class PoseLandmarks:
    left_shoulder, right_shoulder: Point3D
    left_ear, right_ear:           Point3D
    left_elbow, right_elbow:       Point3D
    left_wrist, right_wrist:       Point3D
    left_hip, right_hip:           Point3D
    left_knee, right_knee:         Point3D
    left_ankle, right_ankle:       Point3D
    neck:                          Point3D  # derived midpoint of shoulders
```

Backend → Frontend — WebSocket Payload (JSON)

```
{
  "status": "ok",
  "timestamp": 1716540000.123,
  "exercise": "Squats",
  "dimensions": { "width": 640, "height": 480 },
  "result": {
    "rep_count": 5,
    "rep_just_completed": false,
    "has_issue": false,
    "feedback": "Good form - keep going",
    "metrics": {
      "phase": "descending",
      "left_knee_angle": 145.3,
      "right_knee_angle": 143.7,
      "torso_angle": 12.5,
      "hip_to_knee_ratio": 0.92
    }
  },
  "landmarks": {
    "left_shoulder": { "x": 0.45, "y": 0.30, "z": -0.05 },
    "right_shoulder": { "x": 0.55, "y": 0.30, "z": -0.05 }
  },
  "audio_cue": { "text": "Lower down slowly", "urgent": false }
}
```

Frontend — TypeScript Types

```
interface MaitriFrame {
  timestamp: number;
  exercise: string;
  status: string;                          // "ok" | "no_pose" | "decode_error"
  dimensions: { width: number; height: number };
  result: MaitriResult;
  landmarks: Record<string, Point3D>;
  audio_cue: AudioCue | null;
}
```

```
interface MaitriResult {
  rep_count: number;
  rep_just_completed: boolean;
  has_issue: boolean;
  feedback: string;
  metrics: MaitriMetrics;
}
```

Exercise Plugin Architecture

Every exercise follows the same contract defined in `core/base_analyser.py`:

```
BaseResult (dataclass)              BaseAnalyser (ABC)
──────────────────────              ──────────────────
rep_count: int                      name: str  (class attr)
rep_just_completed: bool            ──────────────────────────
has_issue: bool                     evaluate(landmarks)
feedback: str                         → BaseResult | None
                                    draw_hud(frame, result)
                                      → None
                                    get_audio_cue(result)
                                      → (str, bool) | None
```

Each exercise extends these base classes with its own metrics and registers itself with a single line in `exercises/__init__.py`:

```
EXERCISE_REGISTRY = {
    "Squats":   SquatAnalytics,
    "Push-ups": PushUpAnalytics,
    "Planks":   PlankAnalytics,
}
```

---

4. Pseudocodes

4.1 Backend — Main Processing Pipeline (`backend/pipeline.py`)

```
FUNCTION process_frame(jpeg_bytes, analyser):

    frame ← decode_jpeg(jpeg_bytes)
    IF frame is NULL:
        RETURN { status: "decode_error" }

    landmarks ← MediaPipe.extract_landmarks(frame)

    IF landmarks is NULL:
```

```
        RETURN { status: "no_pose" }

    result ← analyser.evaluate(landmarks)

    audio_cue ← analyser.get_audio_cue(result)

    RETURN {
        status: "ok",
        result: serialise(result),
        landmarks: serialise(landmarks),
        audio_cue: audio_cue
    }
```

4.2 Backend — MediaPipe Pose Extraction (`core/pose_engine.py`)

```
FUNCTION extract_landmarks(frame):

    rgb_frame ← convert_BGR_to_RGB(frame)
    mp_results ← mediapipe_pose.process(rgb_frame)

    IF mp_results.pose_landmarks is NULL:
        RETURN (None, mp_results)

    lms ← mp_results.pose_landmarks.landmark

    FUNCTION get(index):
        RETURN Point3D(lms[index].x, lms[index].y, lms[index].z)

    left_shoulder  ← get(LEFT_SHOULDER)
    right_shoulder ← get(RIGHT_SHOULDER)
    neck ← midpoint(left_shoulder, right_shoulder)

    RETURN PoseLandmarks(
        left_shoulder, right_shoulder, neck,
        left_ear, right_ear,
        left_elbow, right_elbow,
        left_wrist, right_wrist,
        left_hip, right_hip,
        left_knee, right_knee,
        left_ankle, right_ankle
    )
```

4.3 Backend — Squat Analyser (`exercises/squats/analyser.py`)

```
CLASS SquatAnalytics:
    STATE: phase = "standing", rep_count = 0, cue_times = {}
    THRESHOLDS: down_angle=110°, up_angle=160°, depth_angle=90°,
                max_torso_lean=40°, knee_cave_ratio=0.70
```

```
FUNCTION evaluate(landmarks):
    IF landmarks is NULL: RETURN None

    # 1. Compute joint angles
    left_knee_angle  ← angle(left_hip, left_knee, left_ankle)
    right_knee_angle ← angle(right_hip, right_knee, right_ankle)
    avg_knee_angle   ← (left + right) / 2

    # 2. Phase state machine
    rep_just_completed ← False
    SWITCH phase:
        "standing":
            IF avg_knee < down_angle  → phase = "descending"
        "descending":
            IF avg_knee ≤ depth_angle → phase = "bottom"
            IF avg_knee > up_angle    → phase = "standing"   # shallow bail
        "bottom":
            IF avg_knee > down_angle  → phase = "ascending"
        "ascending":
            IF avg_knee ≥ up_angle:
                phase = "standing"
                rep_count += 1
                rep_just_completed = True

    # 3. Form checks
    torso_angle  ← angle(vertical_ref, mid_hip, mid_shoulder)
    forward_lean ← torso_angle > max_torso_lean

    knee_width        ← |left_knee.x - right_knee.x|
    hip_width         ← |left_hip.x  - right_hip.x|
    hip_to_knee_ratio ← knee_width / hip_width
    knee_cave         ← hip_to_knee_ratio < knee_cave_ratio

    depth_reached ← avg_knee_angle ≤ depth_angle
    has_issue     ← knee_cave OR forward_lean

    # 4. Feedback text
    feedback ← build_feedback(knee_cave, forward_lean, phase, rep_count)

    RETURN SquatResult(all fields)

FUNCTION get_audio_cue(result):
    now ← current_time()

    # TIER 1 — Rep completion (urgent)
    IF rep_just_completed:
        CLEAR cue_times
        RETURN ("Rep N complete. Good job!", urgent=True)

    # TIER 2 — Form corrections (urgent, 2s cooldown each)
    IF knee_cave AND (now - cue_times["knee_cave"] ≥ 2.0):
        UPDATE cue_times
        RETURN ("Knees caving in, push them out", urgent=True)
```

```
        IF forward_lean AND (now - cue_times["forward_lean"] ≥ 2.0):
            UPDATE cue_times
            RETURN ("Keep your chest up", urgent=True)

        # TIER 3 — Phase coaching (non-urgent, 6s cooldown, suppressed 3s after
form error)
        IF (now - cue_times["last_form"] < 3.0): RETURN None

        IF (now - cue_times["phase_<current>"] ≥ 6.0):
            RETURN (phase_cue_text[phase], urgent=False)

        RETURN None
```

4.4 Backend — Push-Up Analyser (`exercises/push_ups/analyser.py`)

```
CLASS PushUpAnalytics:
    STATE: phase = "up", rep_count = 0, min_elbow_this_rep = 180°
    THRESHOLDS: up_angle=155°, mid_angle=120°, bottom_angle=90°,
                sag_threshold=0.05, pike_threshold=0.05, flare_ratio=1.30

    FUNCTION evaluate(landmarks):
        # 1. Compute elbow angles (shoulder → elbow → wrist)
        left_elbow_angle  ← angle(left_shoulder, left_elbow, left_wrist)
        right_elbow_angle ← angle(right_shoulder, right_elbow, right_wrist)
        avg_elbow_angle   ← (left + right) / 2

        # 2. Phase state machine + half-rep detection
        SWITCH phase:
            "up":
                min_elbow_this_rep = 180°
                IF avg_elbow < mid_angle → phase = "descending"
            "descending":
                min_elbow = min(min_elbow, avg_elbow)
                IF avg_elbow ≤ bottom_angle → phase = "bottom"
                IF avg_elbow ≥ up_angle:
                    rep_count += 1; rep_just_completed = True
                    half_rep = (min_elbow > bottom_angle)
                    phase = "up"
            "bottom":
                min_elbow = min(min_elbow, avg_elbow)
                IF avg_elbow > mid_angle → phase = "ascending"
            "ascending":
                IF avg_elbow ≥ up_angle:
                    rep_count += 1; rep_just_completed = True
                    half_rep = (min_elbow > bottom_angle)
                    phase = "up"

        # 3. Hip alignment (signed distance of hip from shoulder–ankle line)
        hip_deviation ← signed_body_deviation(hip, shoulder, ankle)  # pick longer
visible side
        sagging_hips  ← hip_deviation >  sag_threshold
```

```
        piking_hips   ← hip_deviation < -pike_threshold

        # 4. Elbow flare (meaningful from front/angled view)
        elbow_width    ← |left_elbow.x - right_elbow.x|
        shoulder_width ← |left_shoulder.x - right_shoulder.x|
        elbow_flare    ← (shoulder_width > 0.05) AND (elbow_width/shoulder_width >
flare_ratio)

        has_issue ← sagging_hips OR piking_hips OR elbow_flare
        RETURN PushUpResult(all fields)
```

4.5 Backend — Plank Analyser (`exercises/planks/analyser.py`)

```
CLASS PlankAnalytics:
    STATE: phase = "resting", rep_count = 0, hold_start = None,
           entry_start = None, best_hold = 0.0
    THRESHOLDS: y_threshold=0.18, entry_grace=2.0s, min_hold=5.0s,
                sag=0.05, pike=0.05, shoulder_sink=0.08, head_drop=0.06

    FUNCTION evaluate(landmarks):
        now ← current_time()

        # 1. Detect horizontal body (plank position)
        shoulder_y ← avg(left_shoulder.y, right_shoulder.y)
        ankle_y    ← avg(left_ankle.y, right_ankle.y)
        body_horizontal ← |shoulder_y - ankle_y| < y_threshold

        # 2. Phase state machine with entry grace period
        rep_just_completed ← False
        SWITCH phase:
            "resting":
                IF body_horizontal:
                    IF entry_start is NULL: entry_start = now
                    IF (now - entry_start) ≥ entry_grace:
                        phase = "in_plank"
                        hold_start = now
                ELSE:
                    entry_start = NULL
            "in_plank":
                IF NOT body_horizontal:
                    hold_dur = now - hold_start
                    best_hold = max(best_hold, hold_dur)
                    IF hold_dur ≥ min_hold:
                        rep_count += 1
                        rep_just_completed = True
                    phase = "resting"; hold_start = NULL

        # 3. Form checks
        hip_deviation    ← signed_body_deviation(hip, shoulder, ankle)
        sagging_hips     ← hip_deviation > sag
        piking_hips      ← hip_deviation < -pike
```

```
        shoulder_sinking ← (shoulder_y - hip_y) > shoulder_sink
        head_dropping    ← (ear_y - shoulder_y) > head_drop

        RETURN PlankResult(all fields, hold_seconds)
```

4.6 Backend — Geometry Helpers (`core/geometry.py`)

```
FUNCTION calculate_angle(p1, p2, p3):
    # Interior angle at vertex p2
    v1 ← p1 - p2
    v2 ← p3 - p2
    dot       ← dot_product(v1, v2)
    cross_mag ← magnitude(cross_product(v1, v2))
    RETURN degrees(arctan2(cross_mag, dot))

FUNCTION signed_body_deviation(hip, shoulder, ankle):
    # Signed perpendicular distance of hip from shoulder–ankle line (2D)
    line_vec ← ankle - shoulder
    line_len ← magnitude(line_vec)
    IF line_len < 1e-6: RETURN 0.0
    RETURN cross_product(line_vec, hip - shoulder) / line_len
    # Positive → hip below line (sagging)
    # Negative → hip above line (piking)
```

4.7 Backend — Audio Feedback (`core/audio.py`)

```
CLASS AudioFeedback:
    queue ← Thread-safe Queue (maxsize=1)
    worker_thread ← daemon Thread

    FUNCTION _worker():          # Runs in background thread
        INIT pyttsx3 engine
        LOOP:
            text ← queue.get()
            IF text is None: BREAK   # shutdown sentinel
            engine.say(text)
            engine.runAndWait()

    FUNCTION say(text):          # Low-priority coaching cue
        IF queue is EMPTY:
            queue.put(text)
        ELSE:
            DISCARD (TTS busy, drop cue)

    FUNCTION say_urgent(text):   # High-priority form correction
        DRAIN queue (remove any pending low-priority cue)
        queue.put(text)          # preempt with correction

    FUNCTION stop():
```

```
        queue.put(None)          # sentinel
        worker_thread.join()
```

4.8 Backend — WebSocket Server (`backend/server.py`)

```
ENDPOINT WS /ws:

    analyser ← None

    LOOP:
        message ← await websocket.receive_text()
        kind    ← message["type"]

        CASE "start":
            exercise ← message["exercise"]
            IF exercise NOT IN EXERCISE_REGISTRY:
                SEND { status: "error" }
            ELSE:
                analyser = EXERCISE_REGISTRY[exercise]()
                SEND { status: "session_started" }

        CASE "frame":
            IF analyser is NULL: SEND { status: "error" }; CONTINUE
            jpeg_bytes ← base64_decode(message["data"])
            payload    ← await run_in_executor(process_frame, jpeg_bytes,
analyser)
            SEND payload

        CASE "stop":
            analyser ← NULL
            SEND { status: "session_stopped" }
```

4.9 Backend — Gemini Rehab Engine (`backend/rehab.py`)

```
FUNCTION get_recommendations(problem_text):

    api_key ← load_env("GEMINI_API_KEY")
    IF api_key is NULL: RAISE ValueError("API key not configured")

    prompt ← build_physio_prompt(problem_text)
    model  ← genai.GenerativeModel("gemini-2.5-flash")
    response ← model.generate_content(prompt)

    text ← strip_markdown_fences(response.text)
    data ← json.parse(text)

    RETURN data["recommendations"]
    # e.g. [{ "exercise": "Squats", "reason": "Strengthens quadriceps..." }]
```

4.10 Frontend — WebSocket Context (`MaitriStreamContext.tsx`)

```
CONTEXT MaitriProvider:
    STATE: availableExercises, currentExercise, isConnected,
           isConnecting, frame, error
    REF: wsRef (WebSocket instance)

    ON_MOUNT:
        response ← HTTP GET /registry
        availableExercises ← response.exercises

    FUNCTION startSession(exercise):
        SET isConnecting = True
        ws ← new WebSocket(WS_URL)

        ws.onopen:
            SET isConnecting = False
            ws.send({ type: "start", exercise })

        ws.onmessage(event):
            payload ← JSON.parse(event.data)
            IF status == "session_started": SET isConnected = True
            IF status == "ok" OR "no_pose":
                SET frame = payload
                IF payload.audio_cue:
                    speakCue(payload.audio_cue.text, payload.audio_cue.urgent)

        ws.onerror: SET error = "Cannot reach Maitri Core Service..."
        ws.onclose: SET isConnected = False

    FUNCTION sendFrame(dataUrl):
        IF ws is OPEN:
            ws.send({ type: "frame", data: dataUrl })

    FUNCTION speakCue(text, urgent):
        IF urgent: synth.cancel()         # preempt current speech
        ELSE IF synth.speaking: RETURN    # drop if busy
        synth.speak(new SpeechSynthesisUtterance(text))
```

4.11 Frontend — Webcam Capture & Frame Loop (`PerformanceHUD.tsx`)

```
COMPONENT PerformanceHUD:
    REF: videoRef (HTMLVideoElement), canvasRef (overlay), captureRef (hidden)

    ON_MOUNT:
        stream ← navigator.mediaDevices.getUserMedia({ video: 640×480 @ 30fps })
        videoRef.srcObject ← stream

    INTERVAL (every 100ms = 10 fps), IF isConnected:
        captureRef.drawImage(videoRef, 0, 0)
```

```
        dataUrl ← captureRef.toDataURL("image/jpeg", quality=0.7)
        sendFrame(dataUrl)

    ON frame UPDATE:
        ctx.clearRect(canvas)
        FOR each connection in POSE_CONNECTIONS:
            pt1 ← landmarks[connection[0]]
            pt2 ← landmarks[connection[1]]
            ctx.drawLine(pt1.x * width, pt1.y * height, ...)
        FOR each landmark:
            ctx.drawCircle(pt.x * width, pt.y * height, radius=5)

    RENDER:
        <video>         ← live webcam feed (mirrored)
        <canvas>        ← skeleton overlay (on top)
        <feedback pill> ← glassmorphic status bar
        border pulsing  ← crimson if has_issue, else zinc
```

---

5. Project Results

Implemented Exercises

Exercise	Phase Detection	Rep Counting	Form Checks	Audio Coaching
Squats	✅ 4-phase FSM	✅ Standing↔Bottom	Knee cave, Forward lean, Depth	✅ 3-tier TTS
Push-ups	✅ 4-phase FSM	✅ Half-rep detection	Hip sag, Hip pike, Elbow flare	✅ 3-tier TTS
Planks	✅ Resting/In-plank	✅ Hold timer (≥5s)	Hip sag, Hip pike, Shoulder sink, Head drop	✅ 3-tier TTS

Squat Analyser — Metrics & Thresholds

Metric	What It Measures	Alert Threshold
`left_knee_angle` / `right_knee_angle`	hip→knee→ankle angle	Phase triggers at 110° / 160°
`torso_angle`	deviation from vertical	> 40° → forward lean alert
`hip_to_knee_ratio`	knee width ÷ hip width	< 0.70 → knee cave alert
`depth_reached`	hip at or below parallel	knee angle ≤ 90°

Push-Up Analyser — Metrics & Thresholds

Metric	What It Measures	Alert Threshold
`elbow_angle`	shoulder→elbow→wrist average	Up: 155°, Bottom: 90°

Metric	What It Measures	Alert Threshold
`hip_deviation`	signed distance hip from plank line	Sag: > 0.05, Pike: < -0.05
`elbow_flare_ratio`	elbow width ÷ shoulder width	> 1.30 with shoulder_width > 0.05
`half_rep`	elbow never reached ≤ 90°	Minimum elbow > 90° in rep

Plank Analyser — Metrics & Thresholds

Metric	What It Measures	Alert Threshold
`hip_deviation`	signed distance hip from shoulder–ankle line	Sag: > 0.05, Pike: < -0.05
`shoulder_sinking`	shoulder Y vs hip Y	(shoulder_y - hip_y) > 0.08
`head_dropping`	ear Y vs shoulder Y	(ear_y - shoulder_y) > 0.06
`hold_seconds`	elapsed hold timer	Counts from entry grace (2s)
`best_hold_seconds`	longest completed hold	Session high-score

Audio Coaching Priority System

All three exercises implement the same **3-tier audio coaching** hierarchy:

Tier	Event	Urgency	Cooldown	Behaviour
1	Rep completed / Hold ended	Urgent	None (clears all timers)	Always fires, resets session
2	Form error (each error type)	Urgent	2–3 seconds per error	Preempts any waiting phase cue
3	Phase coaching	Non-urgent	6–10 seconds per phase	Dropped if TTS is busy; suppressed 3–4s after form error

Frontend UI Results

Feature	Implementation
Real-time skeleton overlay	HTML5 Canvas rendering 12 bone connections at ~10 FPS
Rep counter	Large monospace display with CSS `scale-110` pop animation on completion
Phase indicator	Vertical stepper with active-phase emerald highlight
Live angle charts	Recharts `LineChart` with 300-point rolling window (~10s at 30fps)
Metric gauges	Left/Right knee, torso angle, hip:knee ratio
Feedback pill	Bottom-centered glassmorphic pill over camera; crimson on `has_issue`

Feature	Implementation
Ambient border glow	Pulsing crimson shadow on `has_issue`, neutral zinc otherwise
AI Rehab Screen	Gemini-powered exercise plan from user's problem description
Quick Start	Direct exercise launch from dropdown (no AI required)
Graceful degradation	Connecting overlay + error banners if WebSocket fails

Performance Characteristics

Metric	Value
Frame capture rate (frontend → backend)	~10 FPS
Backend processing rate	~10 FPS (CPU-bound by MediaPipe)
End-to-end latency (capture → display)	~100–200 ms typical
WebSocket payload size	~2–5 KB per frame (JSON + landmarks)
TTS audio delay	< 500 ms (pyttsx3 daemon thread / Web Speech API)
MediaPipe model complexity	Level 1 (balanced accuracy vs. speed)
Webcam capture resolution	640 × 480 @ 30 FPS
JPEG encode quality	0.7 (70%) — reduces WebSocket bandwidth

---

6. Complete Workflow — Frontend to Backend

6.1 Application Startup

```
1. USER runs backend:
   uvicorn backend.server:app --reload --port 8000

2. FastAPI initialises:
   - Loads EXERCISE_REGISTRY from exercises/__init__.py
   - Starts CORS middleware (allows localhost:5173)
   - Registers REST routes: /health, /registry, /recommend
   - Registers WebSocket route: /ws

3. USER runs frontend:
   cd frontend && npm run dev   (Vite dev server → localhost:5173)

4. Browser opens localhost:5173:
   - React mounts MaitriProvider
   - MaitriProvider fires HTTP GET → http://localhost:8000/registry
   - Backend returns: { "exercises": ["Squats", "Push-ups", "Planks"] }
   - State: availableExercises = ["Squats", "Push-ups", "Planks"]
   - App renders RehabScreen (landing page)
```

6.2 Rehab Recommendation Flow (AI Path)

```
USER types problem description → "I have knee pain after running"
USER clicks "Generate My Rehab Plan"

1. Frontend sends:
   POST http://localhost:8000/recommend
   Body: { "problem": "I have knee pain after running" }

2. Backend (rehab.py):
   - Validates problem string (non-empty)
   - Loads GEMINI_API_KEY from .env
   - Builds physiotherapy prompt with exercise list
   - Calls: gemini-2.5-flash.generate_content(prompt)
   - Strips markdown fences from response
   - Parses JSON → list of { exercise, reason }
   - Returns: { "recommendations": [ { "exercise": "Squats", "reason": "..." },
... ] }

3. Frontend renders ExerciseCards:
   - Each card shows exercise name + reason
   - If exercise is in availableExercises → "Start Session" button enabled
   - Click "Start Session" → triggers startSession(exercise)
```

6.3 Quick Start Flow (Direct Path)

```
USER selects exercise from dropdown (pre-populated from /registry)
USER clicks "Quick Start"

→ Calls startSession(selectedExercise) directly
→ Skips AI recommendation entirely
```

6.4 Active Session Initialization

```
startSession("Squats") called in MaitriProvider:

1. State reset:
   - isConnecting = True, frame = null, error = null
   - Previous WebSocket closed (if open)

2. new WebSocket("ws://localhost:8000/ws") opened

3. ws.onopen fires:
   - isConnecting = False
   - ws.send({ "type": "start", "exercise": "Squats" })

4. Backend receives "start":
   - analyser = SquatAnalytics()   ← fresh instance, rep_count=0
```

```
   - ws.send({ "status": "session_started", "exercise": "Squats" })

5. Frontend receives "session_started":
   - isConnected = True
   - App.tsx: isConnected && currentExercise → renders <ActiveSession />

6. ActiveSession renders:
   - PerformanceHUD (2/3 width): webcam + skeleton overlay
   - TelemetryStack (1/3 width): rep counter + metrics + chart

7. PerformanceHUD.useEffect starts webcam:
   - navigator.mediaDevices.getUserMedia({ video: 640×480 @ 30fps })
   - Stream attached to <video> element
```

6.5 Real-Time Frame Processing Loop

```
EVERY 100ms (10 FPS), PerformanceHUD interval fires:

─── FRONTEND ──────────────────────────────────────────────────

1. Capture current video frame:
   captureCanvas.drawImage(videoElement, 0, 0)
   dataUrl = captureCanvas.toDataURL("image/jpeg", 0.7)
   # Result: "..."

2. sendFrame(dataUrl):
   ws.send(JSON.stringify({ type: "frame", data: dataUrl }))

─── BACKEND ───────────────────────────────────────────────────

3. server.py receives "frame" message:
   jpeg_bytes = base64.b64decode(data_url.split(",")[1])
   payload = await run_in_executor(process_frame, jpeg_bytes, analyser)

4. pipeline.process_frame(jpeg_bytes, analyser):

   a. decode_frame(jpeg_bytes):
      arr = np.frombuffer(jpeg_bytes, np.uint8)
      frame = cv2.imdecode(arr, cv2.IMREAD_COLOR)

   b. extract_landmarks(frame):
      rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
      results = mediapipe_pose.process(rgb)
      → Returns PoseLandmarks with 15 joint positions

   c. analyser.evaluate(landmarks):
      [SquatAnalytics example]
      - Computes knee angles via calculate_angle()
      - Runs phase FSM (standing → descending → bottom → ascending → standing)
      - Checks knee cave (hip_to_knee_ratio < 0.70)
      - Checks forward lean (torso_angle > 40°)
```

```
      - Updates rep_count when phase returns to "standing" from "ascending"
      → Returns SquatResult(rep_count, phase, knee_cave, feedback, ...)

   d. analyser.get_audio_cue(result):
      - Checks tier priorities (rep > form > phase)
      - Applies per-cue cooldown timers
      → Returns ("Keep chest up", urgent=True) or None

   e. Serialize payload:
      {
        status: "ok",
        result: { rep_count, rep_just_completed, has_issue, feedback, metrics },
        landmarks: { left_shoulder: {x,y,z}, ... },
        audio_cue: { text: "...", urgent: true } | null
      }

5. ws.send(json.dumps(payload))

─── FRONTEND ──────────────────────────────────────────────────

6. ws.onmessage receives payload:
   payload = JSON.parse(event.data)

   IF status == "ok":
     SET frame = payload
     IF payload.audio_cue:
       speakCue(audio_cue.text, audio_cue.urgent)
         urgent=True  → synth.cancel(); synth.speak(utterance)
         urgent=False → IF !synth.speaking: synth.speak(utterance)

7. React re-renders triggered by frame state change:

   PerformanceHUD:
   - Canvas useEffect fires on frame update
   - ctx.clearRect(canvas)
   - For each POSE_CONNECTION: draw line between landmark[p1] and landmark[p2]
     pt1.x * containerWidth → pixel X
     pt1.y * containerHeight → pixel Y
   - For each landmark: draw circle (radius=5)
   - If has_issue: stroke = "#ef4444" (red), else "#10b981" (emerald)
   - Feedback pill: shows result.feedback text
   - Border glow: crimson pulsing if has_issue

   TelemetryStack:
   - rep_count displayed as large monospace number
   - IF rep_just_completed: setPop(true) → scale-110 animation for 200ms
   - Phase stepper: highlights current phase in emerald
   - MetricGauges: left_knee_angle, right_knee_angle, torso_angle,
hip_to_knee_ratio
   - Chart: appends { time, leftKnee, rightKnee } to rolling 300-point buffer
```

6.6 Session Termination

```
USER clicks "End Session" button in ActiveSession header:

1. Frontend:
   stopSession() called
   ws.send({ type: "stop" })
   ws.close()
   wsRef.current = null
   currentExercise = null, frame = null, isConnected = false
   window.speechSynthesis.cancel()

2. Backend:
   Receives "stop"
   analyser = None  (drops instance, all state cleared)
   ws.send({ status: "session_stopped" })

3. Frontend:
   ws.onclose fires → isConnected = false
   App.tsx: isConnected = false → renders <RehabScreen /> again
   User returns to landing page
```

6.7 Graceful Error Handling

Error Scenario	Backend Response	Frontend Behaviour
WebSocket cannot connect	—	Error banner: "Cannot reach Maitri Core Service..."
Unknown exercise in "start"	`{ status: "error", message: "..." }`	Error state displayed
Frame sent before "start"	`{ status: "error", message: "Send start first." }`	Error state displayed
JPEG decode failure	`{ status: "decode_error" }`	frame = null (no update)
No person detected	`{ status: "no_pose" }`	frame updated, skeleton canvas cleared
Gemini API key missing	HTTP 503	Error card on RehabScreen
Gemini returns bad JSON	HTTP 502	Error card on RehabScreen
WebSocket disconnects mid-session	—	isConnected = false, freeze display, show overlay

6.8 Audio Routing Architecture

Maitri supports two audio routing modes, configurable at runtime:

```
MODE A: Frontend TTS (default — AUDIO_BACKEND = False)
──────────────────────────────────────────────────────
```

```
  Backend pipeline.py → audio_cue field in JSON payload
  → ws.onmessage → speakCue(text, urgent)
  → window.speechSynthesis.speak(utterance)
  ✓ Works headlessly (no SAPI5 required on server)
  ✓ Browser voice quality, multiple language support

MODE B: Backend TTS (AUDIO_BACKEND = True in pipeline.py)
──────────────────────────────────────────────────────────
  Backend pipeline.py → audio.say() / audio.say_urgent()
  → pyttsx3 daemon thread → SAPI5 COM (Windows) → speaker
  audio_cue = null in JSON payload (frontend does not speak)
  ✓ Works in standalone/offline mode
  ✓ Useful when frontend is headless or speech synthesis unavailable
```

Appendix A — How to Add a New Exercise

1. Create `exercises/<name>/analyser.py`:

```python
from core.base_analyser import BaseAnalyser, BaseResult
from dataclasses import dataclass

@dataclass
class LungeResult(BaseResult):
    phase: str
    # add exercise-specific fields

class LungeAnalytics(BaseAnalyser):
    name = "Lunges"

    def evaluate(self, landmarks): ...
    def draw_hud(self, frame, result): ...
    def get_audio_cue(self, result): ...
```

2. Register in `exercises/__init__.py` (one line):

```python
"Lunges": LungeAnalytics,
```

3. No changes required to `main.py`, `backend/server.py`, `backend/pipeline.py`, or any `core/` module.

Appendix B — Running the Project

Start Backend

```
cd C:\Projects\maitri
```

```
.venv\Scripts\uvicorn backend.server:app --reload --port 8000
```

Start Frontend

```
cd C:\Projects\maitri\frontend
npm run dev
```

Desktop Standalone Mode (no browser)

```
cd C:\Projects\maitri
.venv\Scripts\python main.py --webcam --exercise Squats
```

API Quick Reference

Endpoint	Method	Description
`/health`	GET	Liveness probe → `{ "status": "ok" }`
`/registry`	GET	List exercises → `{ "exercises": [...] }`
`/recommend`	POST	Gemini rehab plan → `{ "recommendations": [...] }`
`/ws`	WebSocket	Real-time analysis stream

Documentation generated: 2026-05-24 | Maitri v1.0.0